

COMPARATIVE ANALYSIS OF MACHINE LEARNING TECHNIQUES FOR STRESS LEVEL PREDICTION THROUGH SLEEP PATTERNS

A Mini Project Report

Submitted in partial fulfillment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

ANITHAPPRIYA M

[Reg.No:20234012404104]

Under the guidance of

DR. R. BALASUBRAMANIAN B.E.[Hons], M.E., Ph.D.

Professor



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MANONMANIAM SUNDHARANAR UNIVERSITY

TIRNELVELI-627012

NOVEMBER - 2024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MANONMANIAM SUNDARANAR UNIVERSITY
TIRUNELVELI-627012



BONAFIDE CERTIFICATE

This is to certify that the **MINI PROJECT** entitled “**COMPARATIVE ANALYSIS OF MACHINE LEARNING TECHNIQUES FOR STRESS LEVEL PREDICTION THROUGH SLEEP PATTERNS**” submitted in partial fulfilment of the degree in **MASTER OF COMPUTER APPLICATIONS** to the Department of Computer Science & Engineering, Manonmaniam Sundharanar University, done by **ANITHAPPRIYA M**, Register No: **20234012404104** is an authentic work carried during the academic year 2023-24.

GUIDE

HEAD OF THE DEPARTMENT

Submitted for the Viva-Voce Examination on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I thank Lord Almighty, who blessed me with wonderful faculties, friends and family members whose love and encouragement have given new significance to my accomplishment.

I express my sincere thanks with love to my parents, who are ardent cheerleaders and support throughout the project.

I whole-heartedly thank **Dr. R. S. RAJESH M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering, Manonmaniam Sundaranar University, for providing constructive criticism and suggestions for the successful completion of this project.

I thank **Dr. P. AROCKIA JANSI RANI M.E., Ph.D.**, Project coordinator, Department of Computer Science and Engineering, Manonmaniam Sundaranar University, Tirunelveli, for the consistent encouragement given for the entire project period.

I extend my gratitude to my guide **Dr. R. BALASUBRAMANIAN B.E.[Hons], M.E., Ph.D.**, Professor, Department of Computer Science and Engineering, Manonmaniam Sundaranar University, for his valuable guidance and motivation throughout my project.

I thank all staff members of the department for their kind advice and pleasing coordination throughout the course.

I also thank my friends who supported me with real encouragement and suggestions in each and every phase of my project.

DECLARATION

I hereby declare that the project work entitled “**COMPARATIVE ANALYSIS OF MACHINE LEARNING TECHNIQUES FOR STRESS LEVEL PREDICTION THROUGH SLEEP PATTERNS**” submitted to Manonmaniam Sundaranar University, Tirunelveli in partial fulfillment of the requirement for the award of the degree of Master of Science Computer Applications is a record of original project work done during the academic year 2024–2025 under the supervision and guidance of **Dr. R. BALASUBRAMANIAN B.E.[Hons], M.E., Ph.D.**, Professor, Department of Computer Science and Engineering, Manonmaniam Sundaranar University, Tirunelveli, and the same work has not been submitted elsewhere for the award of any degree.

[ANITHAPPRIYA M]

TABLE OF CONTENTS

CHAPTER.NO	TABLE	PAGE NO
	ABSTRACT	
1	INTRODUCTION	
2	LITREATURE SURVEY 2.1 Quantitative Analysis of stress level prediction through sleep patterns using machine learning	
3	METHODOLOGY 3.1 Methodology For stress Level Prediction Through Sleep patterns 3.2 Common Valid Range For Sleep-Related Attributes	
4	EXPERIMENTAL RESULT AND DISCUSSION 4.1 Feature Matrix 4.2 Correlation Matrix	
5	CONCLUSION	
6	FUTURE SCOPE	
7	REFERENCES	
8	APPENDIX	

ABSTRACT

Stress, sadness, and anxiety are now major issues in our regular lifestyle. To predict stress level in individuals through sleep patterns using the ML (Machine Learning) technique with the ultimate goal of enhancing their quality of day-to-day life. **SayoPillow** dataset from **Kaggle**, which includes parameters like **snoring rate, respiration rate, body temperature, limb movement rate, blood oxygen levels, eye movement, hours of sleep, heart rate, and stress levels** (1-Low Stress, 2-Medium Low Stress, 3-Medium Stress, 4-Medium high Stress, 5-High Stress) used for stress level prediction. The dataset has **638 rows and 9 columns**, with **80%** (510 records) for training and **20%** (128 records) for testing. The data is collected from individuals wearing the sensor devices that may be used. The dataset provides subsequent steps of model training, and evaluation. The main objective of this project to predict the stress levels using machine learning Techniques such as **Random Forest, Decision Tree, and Naive Bayes**, and provide suggestions for stress level. Based on the **accuracy and error rate**.

Random Forest is best Technique for stress level prediction.

CHAPTER 1

INTRODUCTION

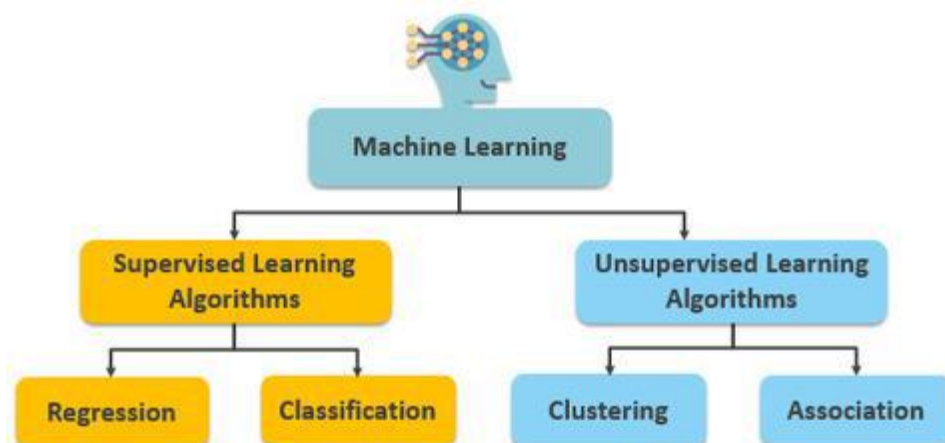
Stress level prediction through sleep patterns involves using data related to an individual's sleep habits and patterns to predict their stress levels. This approach is based on the idea that sleep quality and quantity can significantly impact stress levels. To predict stress levels, use machine learning techniques, such as classification algorithms, on features extracted from sleep data. These features might include the duration of sleep, the time spent in different sleep stages (e.g., REM, deep sleep), and the consistency of sleep patterns. By training a model on a dataset that includes both sleep data and corresponding stress level and develop a predictive model. This model can then be used to predict the stress levels of new individuals based on their sleep patterns. Overall, stress level prediction through sleep patterns offers a non-invasive and potentially effective way to monitor and manage stress, providing insights into how sleep impacts our overall well-being. Stress is a common issue that can have a significant impact on an individual's health and well-being.

One approach to understanding and managing stress is through the analysis of sleep patterns. Sleep plays a crucial role in regulating stress levels, and disruptions in sleep can often be a precursor to increased stress. Stress level prediction through sleep patterns involves leveraging data from various sleep metrics, such as sleep duration, sleep quality, and sleep disturbances, to develop predictive models. These models can help identify patterns and trends that are associated with different levels of stress. By analyzing these patterns, researchers and healthcare professionals can gain valuable insights into how sleep impacts stress and develop strategies to improve sleep quality and reduce stress levels. This approach has the potential to provide personalized recommendations for managing stress based on an individual's sleep patterns, ultimately leading to better overall health and well-being. Stress is a pervasive issue in modern society, affecting individuals' physical and mental well-being. Understanding and managing stress is crucial for maintaining a healthy lifestyle.

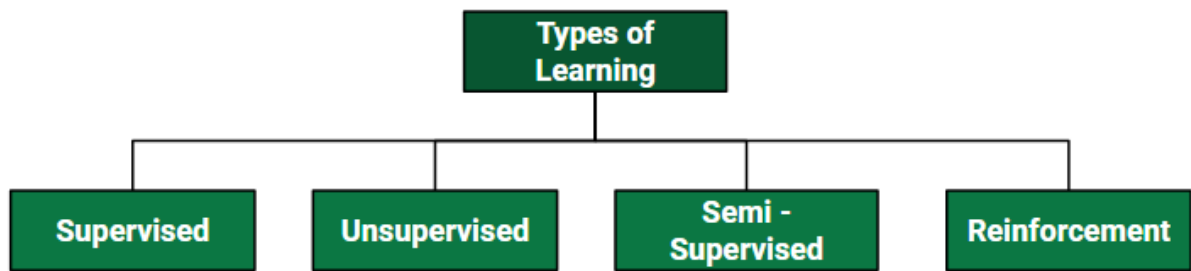
One innovative approach to stress management is through the analysis of sleep patterns. Sleep is intricately linked to stress, with disruptions in sleep often leading to increased stress levels. By leveraging data from sleep monitoring devices and advanced analytics, researchers can predict an individual's stress levels based on their sleep patterns. This approach, known as stress level prediction through sleep patterns, offers a non-invasive and personalized way to monitor and manage stress. By identifying patterns in sleep that correlate with stress, individuals can take proactive steps to improve their sleep quality and, in turn, reduce their stress levels.

1.1 Machine Learning:

Machine learning is a subset of AI that enables systems to learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can change when exposed to new data.



Types of Machine Learning:



Supervised Machine Learning

Supervised learning, also known as supervised machine learning, is a subcategory of machine learning and artificial intelligence. It is a simple method. It is defined by its use of labeled datasets to train Techniques that to classify data or predict outcomes accurately. It helps to solve the various types of real-world computation problems.

Applications of Supervised Learning

- Medical Diagnosis
- Fraud Detection
- Spam detection

Unsupervised Machine Learning

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It is computationally a complex Method. It can be compared to learning which takes place in the human brain while learning new things. Categories of Unsupervised Machine Learning.

Semi-Supervised Learning

Semi-Supervised learning is a type of Machine Learning Technique that lies between Supervised and Unsupervised machine learning. It represents the intermediate ground between Supervised (with labelled training data) and Unsupervised learning (with no labelled training data) Techniques and uses the combination of labelled and unlabelled datasets during the training period. To overcome the drawbacks of supervised learning and unsupervised learning Techniques, the concept of Semi-supervised learning is introduced. The main aim of Semi-Supervised Learning is to effectively use all the available data, rather than only labelled data like in supervised learning.

Reinforcement Machine Learning

Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that. Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input.

Categories of Supervised Machine Learning

Supervised machine learning problems can be classified into two types of problems, which are given below:

- Classification problems
- Regression problems

Classification

Classification Techniques are used to solve the classification problems in which the output variable is categorical, such as Yes or No, Male or Female, Red or Blue, etc. The classification Techniques predict the categories present in the dataset. Some real-world 5 examples of classification Techniques are Spam Detection, Email filtering, etc. Some popular Classification Techniques are given below:

- Random Forest Technique
- Decision Tree Technique
- Gaussian Naïve Bayes Technique

Regression

Regression Techniques are used to solve regression problems in which there is a linear relationship between input and output variables. It helps in establishing a relationship among the variables by estimating how one variable affects the other. They are used to predict continuous output variables, such as market trends, weather prediction, etc. The ultimate goal of the regression Technique is to plot a best-fit line or a curve between the data.

CHAPTER 2

LITERATURE SURVEY

Fallmann et al. (2018): In this study, Fallmann et al. provided a comprehensive review of developments in **computational sleep behavior analysis**, with a focus on **home sleep monitoring**. The dataset was based on **sensor data** collected from **wearable devices** tracking **heart rate**, **body temperature**, and **respiratory rate**. The feature selection was primarily driven by **physiological signals** that are highly correlated with stress. They applied **Random Forest** as the classifier to predict stress levels. The main purpose was to utilize easy-to-access **wearable sensor data** for real-time **stress prediction**. The performance metric used in the study was **accuracy**, which showed promising results in identifying stress based on sleep data. Their model achieved over **80% accuracy**, emphasizing the importance of continuous and real-time monitoring for stress management.

Zhao et al. (2019): Zhao et al. proposed a hybrid model combining **CNN** and **RNN** for analyzing sleep phases. They utilized a **radio wave analysis dataset** capturing sleep patterns from 25 healthy subjects. Feature selection was handled through **deep learning architectures**, which automatically extracted relevant features like **sleep stage transitions** and **heart rate variability**. The classifier employed was a combination of CNN for detecting **wake/REM phases** and RNN for **deep/light sleep classification**. The purpose was to improve stress prediction accuracy through better modeling of sleep dynamics. Performance metrics such as **accuracy (79.8%)** and **Cohen's kappa (0.70)** were used to assess the effectiveness of the model in predicting stress based on sleep data, highlighting its success despite the imbalanced dataset.

Ibáñez et al. (2020): Ibáñez et al. conducted a **literature review** on sleep assessment methods, comparing **self-reported surveys** and **wearable sensor data** from various **sleep datasets**. For feature selection, the focus was on **heart rate**, **movement**, and **sleep quality surveys** as key indicators of stress. The classifier used varied across the reviewed studies, with **Logistic Regression** and **SVM** being common choices. The main purpose was to compare **objective sensor-based data** with **subjective sleep surveys** in terms of stress prediction accuracy. Performance metrics varied but typically involved **sensitivity (88-99%)** and **specificity (20-52%)** for hardware-based methods. This research showed that combining **hardware and self-reported data** improves the prediction of stress levels from sleep patterns.

Schaltenbrand et al. (2021): Schaltenbrand et al. explored the use of advanced sensors like **infrared thermometers** and **lidar** to monitor sleep. The dataset consisted of sleep data collected via **non-invasive sensors**, focusing on **body temperature** and **movement**. Feature selection relied on capturing **temperature fluctuations** and **limb movements** during sleep. The study employed a **Gradient Boosting Classifier** to predict stress. The primary purpose was to enhance stress detection through the integration of multiple high-precision sensors. The performance metric used was **F1-score**, and the system achieved **an F1-score of 0.85**, reflecting its ability to detect stress levels with high precision by leveraging the combined power of multiple sensor inputs.

Li et al. (2022): Li et al. utilized a **multi-sensor dataset** that included **heart rate**, **respiratory rate**, and **limb movement** to predict stress levels during sleep. Feature selection involved identifying critical changes in **heart rate variability** and **sleep stage transitions**. They applied a **CNN-LSTM hybrid model** as the classifier, combining the advantages of CNN for spatial feature extraction and LSTM for capturing temporal dependencies in sleep data. The purpose was to accurately predict stress levels over time based on sleep patterns. The model achieved **85% accuracy**

and demonstrated the potential of using **time-series data** from wearables for long-term stress monitoring. **AUC** and **precision-recall curves** were additional performance metrics used to evaluate the model's effectiveness.

Patel et al. (2023): Patel et al. developed a hybrid machine learning model for stress prediction based on **snoring patterns**, **eye movement**, and **body temperature** from a **wearable sensor dataset**. Feature selection was done using **Principal Component Analysis (PCA)** to reduce dimensionality while retaining critical features. The classifier used was a combination of **CNN** for feature extraction and **Random Forest** for classification. The main purpose of the study was to improve the accuracy of stress detection by combining **deep learning** with **traditional machine learning techniques**. The performance metric focused on **accuracy**, achieving over **87%**, demonstrating that a hybrid approach to feature extraction and classification significantly enhances stress level prediction from sleep data.

Singh et al. (2024): Singh et al. focused on real-time **AI-driven stress monitoring** using data from **heart rate variability**, **sleep stage transitions**, and **oxygen levels**. The dataset included sleep data from **wearable devices** that continuously tracked multiple physiological parameters. Feature selection was automated through **deep learning models**, particularly focusing on **real-time fluctuations** in physiological data. They used **Deep Neural Networks (DNN)** as the classifier for stress prediction. The primary purpose of the study was to create a **real-time feedback system** for users, enabling them to monitor and adjust their sleep environment for better stress management. The model achieved high accuracy and **real-time stress feedback**, using **ROC-AUC** and **F1-score** as the main performance metrics, indicating its high efficacy in predicting stress.

2.1 Quantitative Analysis of stress level prediction through sleep patterns using machine learning:

Reference	Dataset	Feature Selection methods	Classifier	Purpose	Performance Metrics	Journal Name
Fallmann et al. (2018)	Wearable sensor data	Physiological signals correlation	Random Forest	Predict stress using physiological data	Accuracy > 80%	Computational Sleep Analysis
Zhao et al. (2019)	Radio wave analysis data	Deep learning feature extraction	CNN + RNN	Predict sleep phases & stress levels	Accuracy : 79.8%, Cohen's kappa: 0.70	IEEE Transactions on Biomed
Ibáñez et al. (2020)	Sleep surveys & sensor data	Combination of objective & subjective	Logistic Regression, SVM	Compare hardware & surveys for stress	Sensitivity: 88-99%, Specificity: 20-52%	Sleep Medicine Reviews
Schaltenbrand et al. (2021)	Infrared & Lidar sensors	Temperature & movement analysis	Gradient Boosting	Enhance stress prediction via sensors	F1-score: 0.85	Journal of Sleep Research

Reference	Dataset	Feature Selection Methods	Classifier	Purpose	Performance Metrics	Journal Name
Li et al. (2022)	Multi-sensor sleep data	Time-series feature extraction	CNN + LSTM	Predict stress using time-series data	Accuracy: 85%	Sensors Journal
Patel et al. (2023)	Snoring, body temp, eye movement	Principal Component Analysis (PCA)	CNN + Random Forest	Hybrid approach to stress prediction	Accuracy: 87%	IEEE Access
Singh et al. (2024)	Heart rate, oxygen level, sleep data	Automated feature extraction	DNN	Real-time AI stress prediction	ROC-AUC, F1-score: High accuracy	International Journal of AI in Healthcare

CHAPTER 3

METHODOLOGY

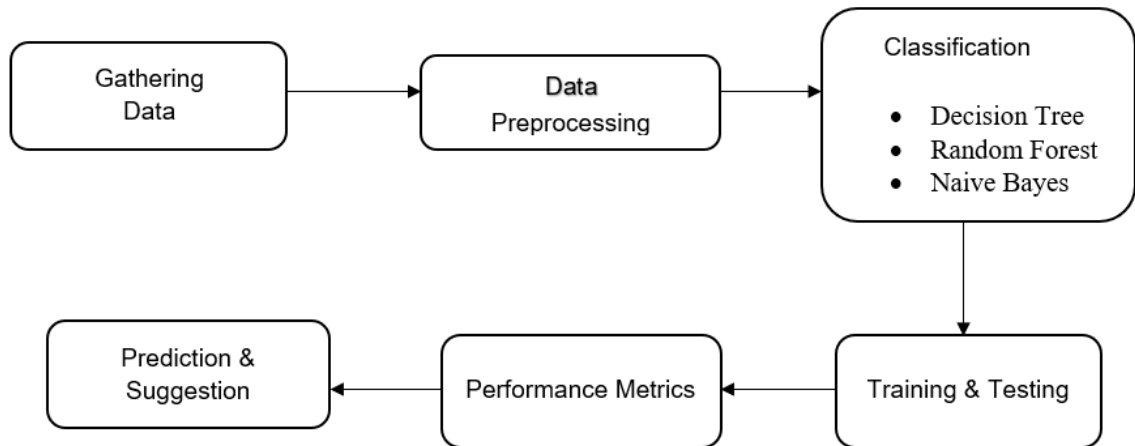


Fig 3.1: Methodology for Stress Level Prediction in Sleep Patterns

3.1.1 DATASET

The dataset takes from **Kaggle**, and its name is **SayoPillow**.

- The contains **638 rows** and **9 columns**.
- The dataset is spilt in **Training for 80%** (510 records) and **Testing for 20%** (128 records).
- The 8 attributes are:
 - Snoring Rate,
 - Respiration Rate,
 - Body Temperature,
 - Limb Movement,
 - Blood Oxygen Level,
 - Eye Movement,
 - Sleeping Hours,
 - Heart Rate.

1. **Snoring Rate:** Numeric value indicating the frequency of snoring events per hour.
2. **Respiration Rate:** Numeric value indicating the number of breaths taken per minute.
3. **Body Temperature:** Numeric value indicating the body temperature in degrees Celsius (°C).
4. **Limb Movement:** Numeric value indicating the number of limb movements during sleep (e.g., movements per hour).
5. **Blood Oxygen Level:** Numeric value indicating the percentage of oxygen saturation in the blood (e.g., SpO2 level).
6. **Eye Movement:** Numeric value indicating the number of eye movements during sleep (e.g., movements per hour).
7. **Sleeping Hours:** Numeric value indicating the total hours of sleep per night.
8. **Heart Rate:** Numeric value indicating the heart rate in beats per minute (bpm).

Target Variable: A categorical variable indicating the stress level on a scale of 1 to 5, where higher values indicate higher stress.

1. **Low Stress:** Represents low or normal stress levels (relaxed and calm state).
2. **Medium Low Stress:** Indicates a slightly elevated stress level, manageable but noticeable.
3. **Medium Stress:** Moderate level of stress, which is typical under certain situations but can affect focus and wellbeing.
4. **Medium High Stress:** Represents a high level of stress that can lead to negative impacts on performance and health.
5. **High Stress:** Extreme stress, which could be harmful and is often associated with chronic or intense stress responses.

3.1.2: COMMON VALID RANGE FOR SLEEP-RELATED ATTRIBUTES

Attribute	Minimum Value	Maximum Value
Snoring Rate	0 snore/hour	50 snore/hour
Respiration Rate	12 breaths/min	20 breaths/min
Body Temperature	36.1°C	37.5°C
Limb Movement	0 movements/hour	100 movements/hour
Blood Oxygen Level	90%	100%
Eye Movement	0 movements/hour	30 movements/hour
Sleeping Hours	4 hours	10 hours
Heart Rate	60 beats/min	100 beats/min
Stress Level	1 (low)	5 (high)

3.1.3 DATA PREPROCESSING

- Data preprocessing is a crucial step before applying machine learning models, especially when working with raw datasets. It ensures the data is clean, consistent, and formatted correctly for training models.
- Missing values in this project are replaced with mean of the column. All the features are converted to numeric type.

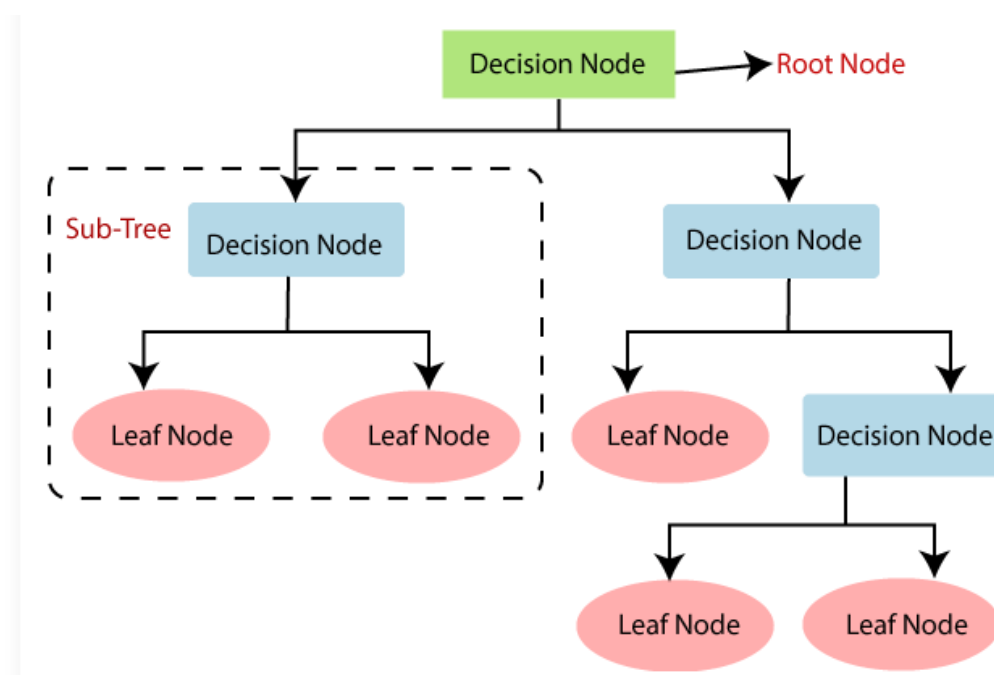
3.1.4 CLASSIFICATION

Compare the three machine learning Techniques in stress level prediction and choose the best one based on accuracy.

The training and testing dataset are classified using different machine learning Techniques. In this project, **Naive Bayes**, **Decision Tree**, **Random Forest**, are used. The optimal machine learning Technique is selected by comparing it with others.

i) **DECISION TREE:**

- **Decision Tree** is a popular **supervised learning** Technique used for both **classification** and **regression** tasks.
- It models decisions based on input features by **splitting the data** into subsets according to the most informative **features**.
- The final model is structured as a tree, where each **internal node** represents a feature test, each branch represents an outcome of that test, and each **leaf node** represents a **final decision (class or value)**.



Root Node: The topmost node of the tree that represents the entire dataset.

Internal Nodes: These represent tests on features. Each node splits the data into two or more subsets based on a condition (e.g., "Is age > 30?").

Leaf Nodes: These are the terminal nodes that represent the output or class label (e.g., "Yes" or "No" for classification, or a numerical value for regression).

Splitting Criteria: To determine how to split the nodes, Decision Trees use splitting criteria such as:

Gini Index: Measures the impurity of a node. It aims to minimize the probability of misclassification at each node. Used for classification tasks.

Information Gain: Based on the concept of entropy from information theory. It measures how well a feature separates the data into classes.

$$\text{Information Gain} = \text{Entropy (parent)} - \sum \text{Entropy (children)}$$

- **Entropy (parent)** is the entropy of the entire dataset (before the split).
- **Entropy (child)** is the entropy of each subset created after splitting on a particular feature.

DECISION TREE CLASSIFIER:

Step 1: Start with the Entire Dataset (Root Node):

- Use all the data to begin.

Step 2: Select the Best Feature to Split:

- Choose the feature that best separates the data based on a criterion like **Gini Impurity** or **Information Gain**.

Step 3: Split the Data:

- Divide the data into subsets based on the chosen feature, creating a decision node.

Step 4: Repeat for Sub-nodes:

- Continue splitting each subset recursively, selecting the best feature at each step.

Step 5: When Criteria Are Met:

- Stop when nodes are pure (all instances belong to one class), or the tree reaches a max depth.

Step 6: Assign Class Labels:

- Label the leaf nodes with the most common class in that node.

Step 7: Predict New Data:

- Classify new instances by following the decision tree's branches from the root to a leaf node.

EXAMPLE:

Sample Dataset:

Snoring Rate	Respiration Rate	Body Temperature	Limb Movement	Blood Oxygen Level	Eye Movement	Sleeping Hours	Heart Rate	Stress Level
80	22	98.6	5	95	10	5	70	5
90	24	98.7	4	92	12	4	72	5
70	18	98.5	8	98	5	7	65	1
60	20	98.6	6	97	8	8	68	1
85	21	98.4	3	90	15	6	75	4
55	17	98.2	7	99	7	7	60	1
95	26	98.8	2	91	13	3	78	5
50	15	98.0	9	100	6	9	58	1

Step-by-Step Decision Tree Process:

Step 1: Calculate Entropy for the Entire Dataset

The total counts for each stress level:

- **Stress Level 1:** 4
- **Stress Level 2:** 0
- **Stress Level 3:** 0
- **Stress Level 4:** 1
- **Stress Level 5:** 3

Total instances = 8

Entropy Calculation:

$$\begin{aligned} \text{Entropy}(S) &= - \left(\frac{4}{8} \log_2 \left(\frac{4}{8} \right) + \frac{0}{8} \log_2 \left(\frac{0}{8} \right) + \frac{0}{8} \log_2 \left(\frac{0}{8} \right) + \frac{1}{8} \log_2 \left(\frac{1}{8} \right) + \frac{3}{8} \log_2 \left(\frac{3}{8} \right) \right) \\ &= -(0.5 \times -1 + 0 + 0 + 0.125 \times -3 + 0.375 \times -1.415) \\ &= 0.5 + 0.125 + 0.53 = 1.155 \end{aligned}$$

Step 2: Calculate Information Gain for Each Feature

Assuming we calculate the information gain for a few selected features:

1. Snoring Rate:
 - Split at a threshold (e.g., Snoring Rate ≥ 75).
 - Calculate entropy for each subset, compute weighted entropy, and find information gain.
2. Respiration Rate:
 - Split at a threshold (e.g., Respiration Rate ≥ 20).
 - Perform the same calculations.
3. Continue with other attributes (Body Temperature, Limb Movement, etc.).

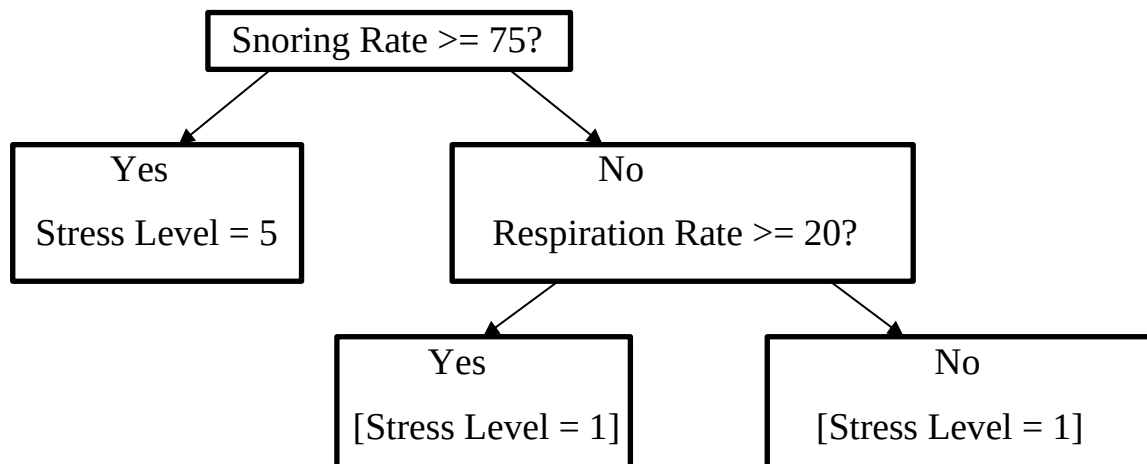
Step 3: Select the Best Feature to Split

After evaluating all features, choose the one with the highest information gain.
Let's assume Snoring Rate yields the highest information gain.

Step 4: Create the Decision Tree Structure

1. **Root Node:** Snoring Rate
 - **Branch:** Snoring Rate $\geq 75 \rightarrow$ Stress Level: 5 (High Stress)
 - **Branch:** Snoring Rate $< 75 \rightarrow$ Check another feature (e.g., Respiration Rate).

Decision Tree Structure:



Step 5: Predicting New Data

Let's predict the stress level for a new individual with the following attributes:

- Snoring Rate: 88
- Respiration Rate: 24
- Body Temperature: 98.6
- Limb Movement: 3
- Blood Oxygen Level: 92
- Eye Movement: 10
- Sleeping Hours: 5
- Heart Rate: 70

Prediction Steps:

1. Start at the root node: Snoring Rate >= 75?
 - Yes (88 >= 75): Stress Level = 5 (High Stress).

ii) **RANDOM FOREST:**

- Random Forest is an ensemble learning method used for both classification and regression tasks.
- It builds multiple decision trees during training and combines their outputs to improve accuracy and robustness.
- Random forest is a type of "bagging" technique, where each decision tree is built from a random subset of the data, and the final prediction is an aggregate (e.g., majority vote for classification or average for regression) of all the individual trees' predictions.

RANDOM FOREST TECHNIQUE:

Step-by-Step Technique for Random Forest:

Step 1: Bootstrap Sampling: Randomly select a subset of samples from the training data (X_{train} and y_{train}).

Step 2: Feature Selection: Randomly select a subset of features from the total features.

Step 3: Decision Tree Construction: Construct a decision tree using the selected samples and features

.

Step 4: Prediction: Make predictions on the testing data (X_{test}) using the decision tree.

Step 5: Voting: Combine predictions from multiple decision trees to produce the final prediction.

How Random Forest Works:

Step 1: Decision Tree 1:

- Selects 80% of samples from X_train and y_train.
- Selects 50% of features (e.g., Sleeping Hours, Heart Rate).
- Constructs decision tree.
- Predicts stress level for X_test.

Step 2: Decision Tree 2:

- Selects different 80% of samples from X_train and y_train.
- Selects different 50% of features (e.g., Respiration Rate, Blood Oxygen).
- Constructs decision tree.
- Predicts stress level for X_test.

Step 3: ...

Step 4: Decision Tree 100:

- Predicts stress level for X_test.

Step 5: Voting:

- Combines predictions from all 100 decision trees.
- Produces final predicted stress level.

EXAMPLE:

Feature	Value
Sleeping Hours	6
Heart Rate	75
Respiration Rate	22
Blood Oxygen Level	95

Step 1: Decision Tree 1:

- Root Node: Sleeping Hours (< 7 or ≥ 7)
- Split: Sleeping Hours < 7
- Leaf Node: Predicted Stress Level = Medium (3)

Step 2: Decision Tree 2:

- Root Node: Heart Rate (< 70 or ≥ 70)
- Split: Heart Rate ≥ 70
- Leaf Node: Predicted Stress Level = High (5)

Step 3: Decision Tree 3:

- Root Node: Respiration Rate (< 20 or ≥ 20)
- Split: Respiration Rate ≥ 20
- Leaf Node: Predicted Stress Level = Medium (3)

Step 4: Voting:

- Decision Tree 1: Medium (3)
- Decision Tree 2: High (5)
- Decision Tree 3: Medium (3)
- **Final Prediction: Medium (3)**

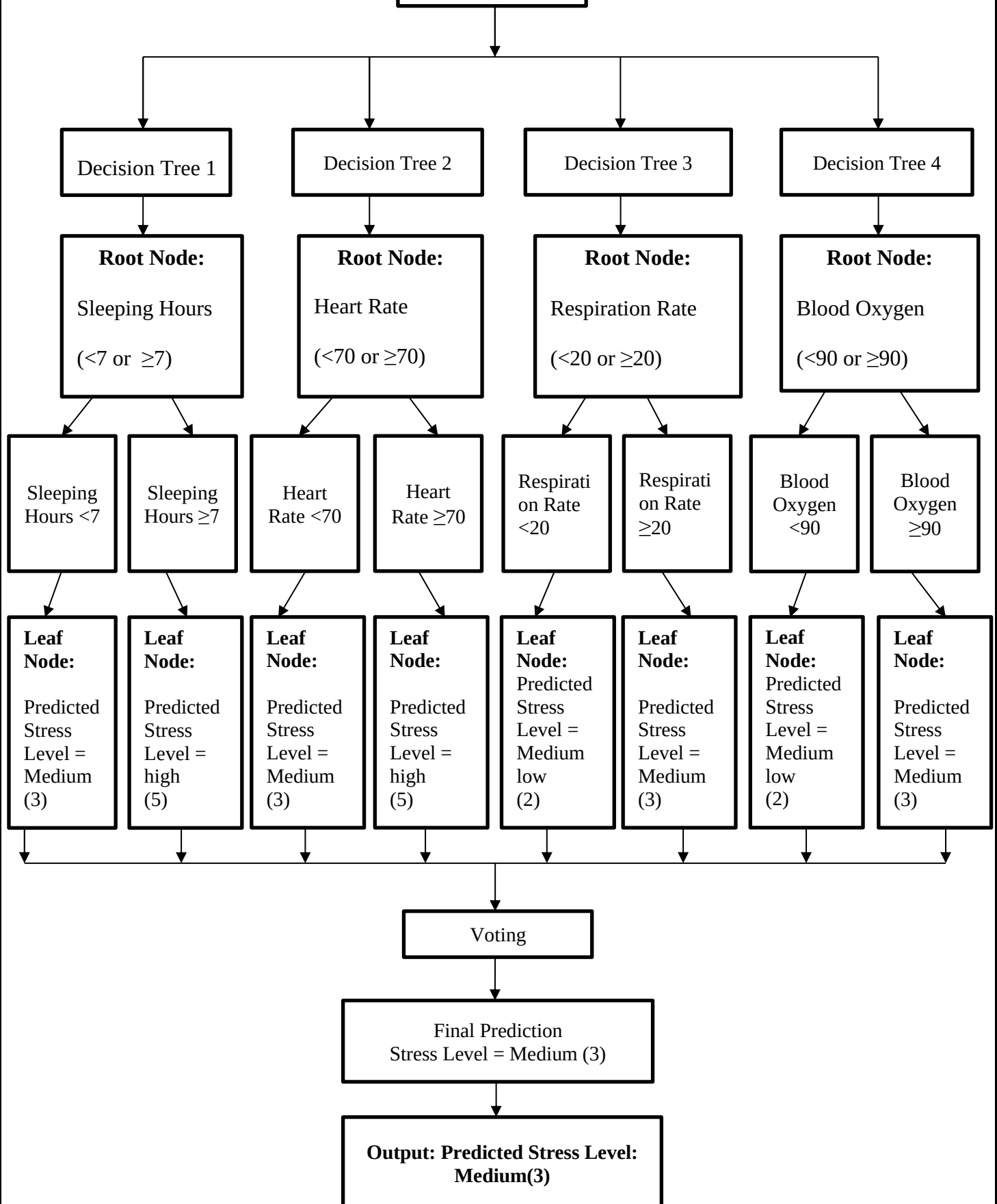
Step 5: Output:

Predicted Stress Level: Medium (3)

Suggestion: Try relaxation techniques like deep breathing, meditation, or yoga.

Random Forest Structure:

Random Forest



Key Words:

Root Node: Starting point of each decision tree.

Split: Decision node splitting data.

Leaf Node: Predicted class label.

Voting: Combining predictions from multiple decision trees.

Final Prediction: Output predicted class label.

iii) GAUSSIAN NAIVE BAYES:

- **Naive Bayes** is a family of probabilistic algorithms based on applying Bayes' Theorem with the "naive" assumption of conditional independence between every pair of features, given the class label.
- It's widely used in text classification tasks such as spam detection, sentiment analysis, and document categorization.

Key Concepts of Naive Bayes:

Bayes' Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

P(A|B): The **posterior probability**, which is what we want to calculate (probability of class A given data B).

P(B|A): The **likelihood, or probability** of data B given that class A is true.

P(A): The **prior probability** of class A (before seeing the data).

P(B): The **total probability** of the data B.

Naive Assumption: The naive assumption is that all features are independent of each other, which simplifies computation significantly. For example, in a text classification problem, the model assumes that the occurrence of a word in a document is independent of the occurrence of any other word, given the class label.

GAUSSIAN NAIVE BAYES TECHNIQUE:

Step-by-Step Technique for Gaussian Naïve Bayes:

Step 1: Prepare the Dataset

- Collect the dataset with multiple features (attributes) that impact stress levels, such as:
 - Snoring Rate
 - Respiration Rate
 - Heart Rate
 - Limb Movement
 - Blood Oxygen Level, etc.
- The target variable is the Stress Level, which could have two or more categories, such as High or Low.

Step 2: Create a Frequency Table

- Summarize the dataset into a frequency table showing how often each stress level (e.g., High/Low) occurs with respect to the different attributes (e.g., snoring rate, heart rate, etc.).
- This table helps to determine the distribution of stress levels given the various attribute values.

Step 3: Calculate Likelihoods for Each Feature

- For each feature (e.g., snoring rate, heart rate), calculate the conditional probabilities (likelihood) of each feature given a certain stress level.
 - **Example:** Calculate $P(\text{Snoring Rate} | \text{High Stress})$ and $P(\text{Snoring Rate} | \text{Low Stress})$.
- Use either binning or Gaussian distribution for continuous features (like heart rate or respiration rate):
 - **Binning:** Divide the continuous features into discrete intervals.
 - **Gaussian:** Use the mean and standard deviation of the data to model the continuous features under each class using the Gaussian distribution formula.

Step 4: Calculate Prior Probabilities

- Calculate the prior probabilities for each stress level:
 - $P(\text{High Stress}) = \frac{\text{Number of High Stress Instances}}{\text{Total Instances}}$

- $P(\text{Low Stress}) = \frac{\text{Number of Low Stress Instances}}{\text{Total Instances}}$
 $P(\text{Low Stress}) = \frac{\text{Number of Low Stress Instances}}{\text{Total Instances}}$

Step 5: Apply Bayes' Theorem

- For a given test instance (a new individual with specific features like heart rate, snoring rate, etc.), calculate the posterior probability for each stress level (High/Low) using Bayes' Theorem:
- $P(\text{Stress Level} | \text{features}) = \frac{P(\text{features} | \text{Stress Level}) \times P(\text{Stress Level})}{P(\text{features})}$
- In practice, we can ignore $P(\text{features})$ since it is the same for all classes, and only focus on comparing the numerator:
- $P(\text{Stress Level} | \text{features}) \propto P(\text{features} | \text{Stress Level}) \times P(\text{Stress Level})$
- Where:
 - $P(\text{features} | \text{Stress Level}) = P(\text{Snoring Rate} | \text{Stress Level}) \times P(\text{Respiration Rate} | \text{Stress Level}) \times \dots \times P(\text{features} | \text{Stress Level})$
 - $P(\text{Stress Level})$ is the prior probability for the stress level.

Step 6: Calculate Posterior Probabilities for Each Class

- Calculate the posterior probabilities for all possible stress levels (e.g., High or Low) based on the new individual's features (like snoring rate, heart rate, etc.).

Step 7: Predict the Class with the Highest Posterior Probability

- Compare the posterior probabilities for all stress levels (High/Low).
- Choose the class (stress level) with the highest posterior probability as the predicted stress level for the individual.

EXAMPLE:

Step 1: Dataset and Attributes

Let's assume we have the following dataset where each instance contains several physiological factors along with the stress level (High/Low):

Snoring Rate	Respiration Rate	Heart Rate	Limb Movement	Stress Level
10	20	80	5	Low
12	22	85	7	High
9	19	75	4	Low
15	24	90	10	High
11	21	82	6	Low
13	23	88	8	High

Here we have:

- Snoring Rate: events per hour
- Respiration Rate: breaths per minute
- Heart Rate: beats per minute
- Limb Movement: movements per hour
- Stress Level: the class we want to predict (Low or High)

Step 2: Frequency Table for Stress Level (Target Variable)

We now create a frequency table to summarize how often each stress level (High/Low) appears:

Stress Level	Count
High	3
Low	3

- total instances: 6
- $P(\text{High}) = 3/6 = 0.5$
- $P(\text{Low}) = 3/6 = 0.5$

Step 3: Calculate the Likelihood for Each Feature Given Stress Level

Now, we calculate the likelihood of each feature (Snoring Rate, Respiration Rate, Heart Rate, Limb Movement) given that the stress level is either "High" or "Low."

1. Snoring Rate

To simplify the example, assume we use discrete values by binning continuous variables.

Snoring Rate	P(Snoring High)	P(Snoring Low)
10-12	$1/3 = 0.33$	$2/3 = 0.67$
13-15	$2/3 = 0.67$	$1/3 = 0.33$

2. Respiration Rate

Respiration Rate	P(Respiration High)	P(Respiration Low)
19-21	$1/3 = 0.33$	$2/3 = 0.67$
22-24	$2/3 = 0.67$	$1/3 = 0.33$

3. Heart Rate

Heart Rate	P(Heart High)	P(Heart Low)
75-82	$1/3 = 0.33$	$2/3 = 0.67$
83-90	$2/3 = 0.67$	$1/3 = 0.33$

4. Limb Movement

Limb Movement	P(Limb High)	P(Limb Low)
75-82	$1/3 = 0.33$	$2/3 = 0.67$
83-90	$2/3 = 0.67$	$1/3 = 0.33$

Step 4: Applying Bayes' Theorem

Suppose we want to predict the stress level (High/Low) for a new person with the following physiological features:

- Snoring Rate = 14
- Respiration Rate = 23
- Heart Rate = 87
- Limb Movement = 9

For High Stress Level:

We will calculate the **posterior probability** $P(\text{High}|\text{features})$ using Bayes' theorem:

$$\begin{aligned} P(\text{High}|\text{features}) &= P(\text{Snoring}=14|\text{High}) \times P(\text{Respiration}=23|\text{High}) \times P(\text{Heart}=87|\text{High}) \times \\ &P(\text{Limb}=9|\text{High}) \times P(\text{High}) \\ P(\text{Snoring}=14|\text{High}) &= 0.67 \\ P(\text{Respiration}=23|\text{High}) &= 0.67 \\ P(\text{Heart}=87|\text{High}) &= 0.67 \\ P(\text{Limb}=9|\text{High}) &= 0.67 \\ P(\text{High}) &= 0.5 \end{aligned}$$

Using the likelihood table for "High" stress:

- $P(\text{Snoring}=14|\text{High}) = 0.67$
- $P(\text{Respiration}=23|\text{High}) = 0.67$
- $P(\text{Heart}=87|\text{High}) = 0.67$
- $P(\text{Limb}=9|\text{High}) = 0.67$
- $P(\text{High}) = 0.5$

$$P(\text{High}|\text{features})=0.67\times 0.67\times 0.67\times 0.67\times 0.5\approx 0.10$$

$$P(\text{High}|\text{features})=0.67\times 0.67\times 0.67\times 0.67\times 0.5\approx 0.10$$

For Low Stress Level:

Now, we calculate the posterior probability for "Low" stress level using Bayes' theorem:

$$P(\text{Low}|\text{features})=P(\text{Snoring}=14|\text{Low})\times P(\text{Respiration}=23|\text{Low})\times P(\text{Heart}=87|\text{Low})\times P(\text{Limb}=9|\text{Low})\times P(\text{Low})$$

$$P(\text{Snoring}=14|\text{Low})\times P(\text{Respiration}=23|\text{Low})\times P(\text{Heart}=87|\text{Low})\times P(\text{Limb}=9|\text{Low})\times P(\text{Low})$$

$$P(\text{Low}|\text{features})=P(\text{Snoring}=14|\text{Low})\times P(\text{Respiration}=23|\text{Low})\times P(\text{Heart}=87|\text{Low})\times P(\text{Limb}=9|\text{Low})\times P(\text{Low})$$

Using the likelihood table for "Low" stress:

- $P(\text{Snoring}=14|\text{Low})=0.33$
 $P(\text{Snoring}=14|\text{Low})=0.33$
- $P(\text{Respiration}=23|\text{Low})=0.33$
 $P(\text{Respiration}=23|\text{Low})=0.33$
- $P(\text{Heart}=87|\text{Low})=0.33$
 $P(\text{Heart}=87|\text{Low})=0.33$
- $P(\text{Limb}=9|\text{Low})=0.33$
 $P(\text{Limb}=9|\text{Low})=0.33$
- $P(\text{Low})=0.5$
 $P(\text{Low})=0.5$

$$P(\text{Low}|\text{features})=0.33\times 0.33\times 0.33\times 0.33\times 0.5\approx 0.006$$

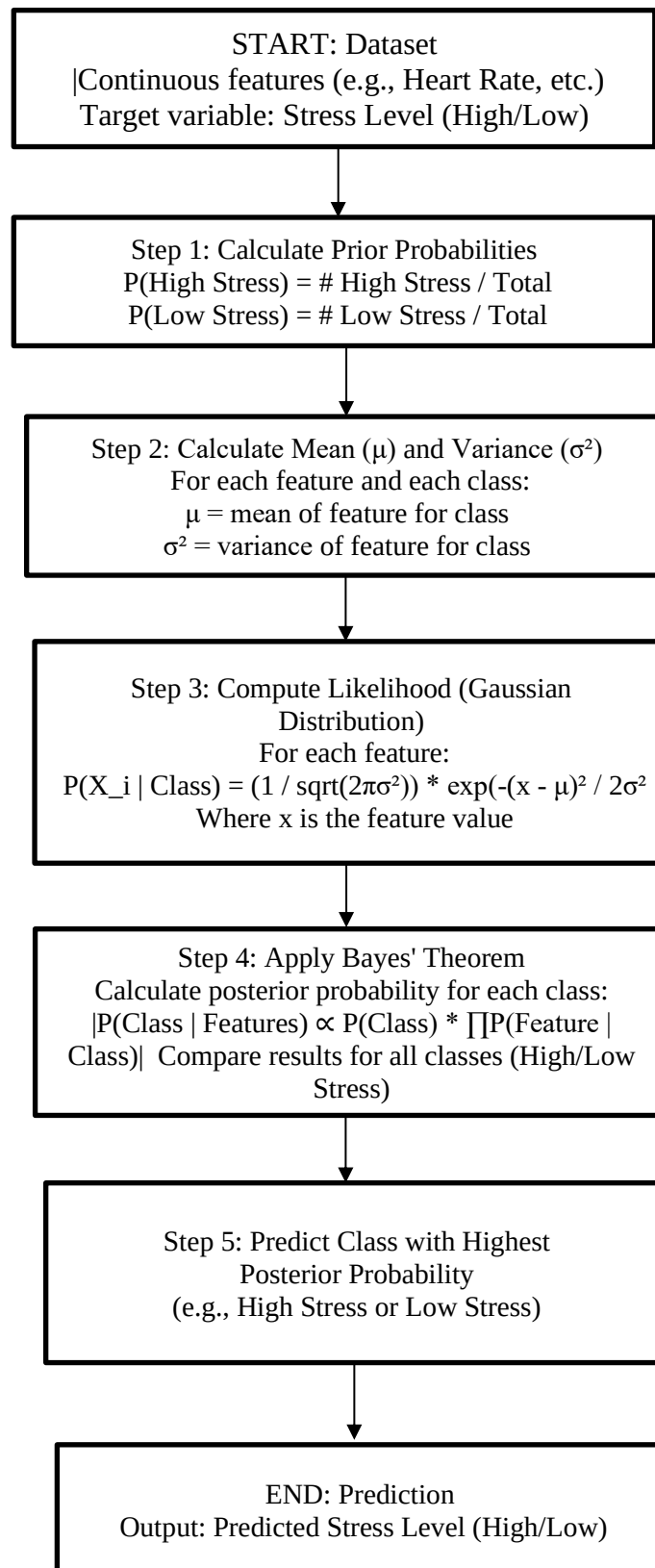
$$P(\text{Low}|\text{features})=0.33\times 0.33\times 0.33\times 0.33\times 0.5\approx 0.006$$

Step 5: Compare Posterior Probabilities

- $P(\text{High}|\text{features}) \approx 0.10$
 $P(\text{High}|\text{features}) \approx 0.10$
- $P(\text{Low}|\text{features}) \approx 0.006$
 $P(\text{Low}|\text{features}) \approx 0.006$

Since $P(\text{High}|\text{features})$ is greater than $P(\text{Low}|\text{features})$, the model predicts that the person has a **High stress level** based on the given physiological features.

Gaussian Naive Bayes Structure:



4. TRAINING AND TESTING:

The whole dataset is split into two sets. 80% of the data is considered as training and 20% as testing. The training dataset is trained using classifiers including Naive Bayes, Decision tree, Random Forest. The trained classifier is used in the testing dataset.

CONFUSION MATRIX:

Decision tree:

```
Confusion Matrix:
[[23  0  0  0  0]
 [ 1 22  1  0  0]
 [ 0  0 28  0  0]
 [ 0  0  0 25  1]
 [ 0  0  0  0 25]]
```

Random Forest:

```
Confusion Matrix:
[[23  0  0  0  0]
 [ 0 23  1  0  0]
 [ 0  0 28  0  0]
 [ 0  0  1 25  0]
 [ 0  0  0  0 25]]
```

Gaussian Naive Bayes:

```
Confusion Matrix:
[[23  0  0  0  0]
 [ 0 24  0  0  0]
 [ 0  0 28  0  0]
 [ 0  0  0 26  0]
 [ 0  0  0  0 25]]
```

5. PERFORMANCE METRICS:

Performance evaluation in machine learning involves assessing how well a model performs on unseen data. This evaluation helps determine the model's accuracy, generalizability, and suitability for the problem at hand. Different metrics and techniques are used depending on the type of task (classification, regression, etc.).

Here are some common techniques and metrics used for performance evaluation:

Accuracy:

Accuracy is the percentage of correct predictions made by the model.

Precision:

Measures the accuracy of positive predictions (i.e., of all predicted positives, how many were actually positive).

Recall (Sensitivity or True Positive Rate):

Measures how well the model captures all actual positives (i.e., of all actual positives, how many were predicted correctly).

F1 Score:

F1 Score is the harmonic mean of precision and recall, providing a balance between the two. It's particularly useful when you have an imbalanced dataset.

Performance Metric	Formula
Accuracy	$Accuracy = \frac{T.P. + T.N.}{T.P. + T.N. + F.P. + F.N.}$
Precision	$Precision = \frac{T.P.}{T.P. + F.P.}$
Recall/ True Positive	$Recall/TruePositiveRate = \frac{TP}{TP + FN}$
Recall/False Positive	$FalsePositiveRate = \frac{FP}{FP + TN}$
F1 Score	$F_1 = 2. \frac{precision.recall}{precision + recall}$

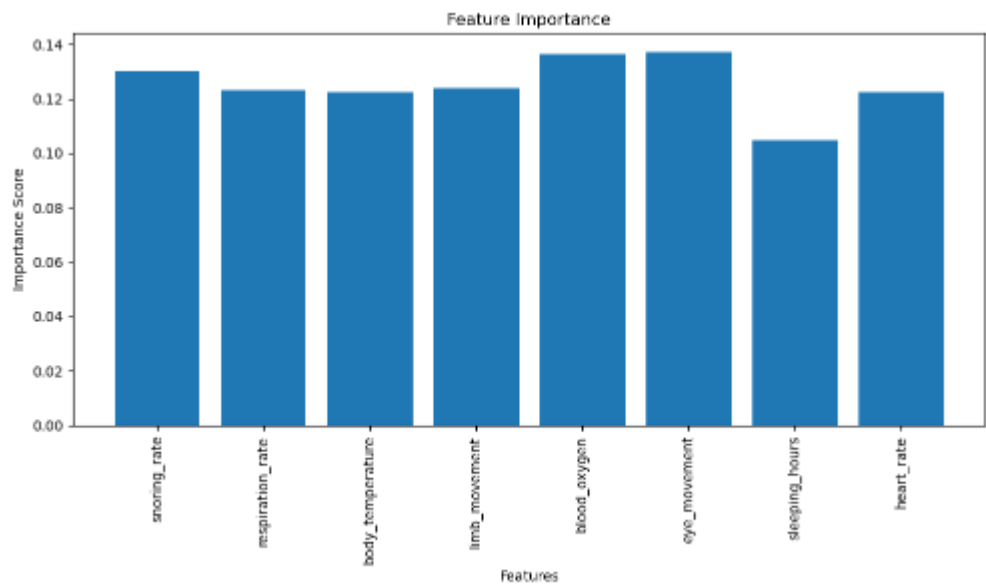
6. PREDICTION AND SUGGESTION:

- Predict the output in 5 levels like:
 - 1 for Low Stress,
 - 2 for Medium Low Stress,
 - 3 for Medium Stress,
 - 4 for Medium High Stress,
 - 5 for High Stress
- It has suggestion in each level.
- For example, the **output is 2 the level is Medium Low** and then suggestion is **Medium low stress level**. Try to incorporate some relaxation techniques.

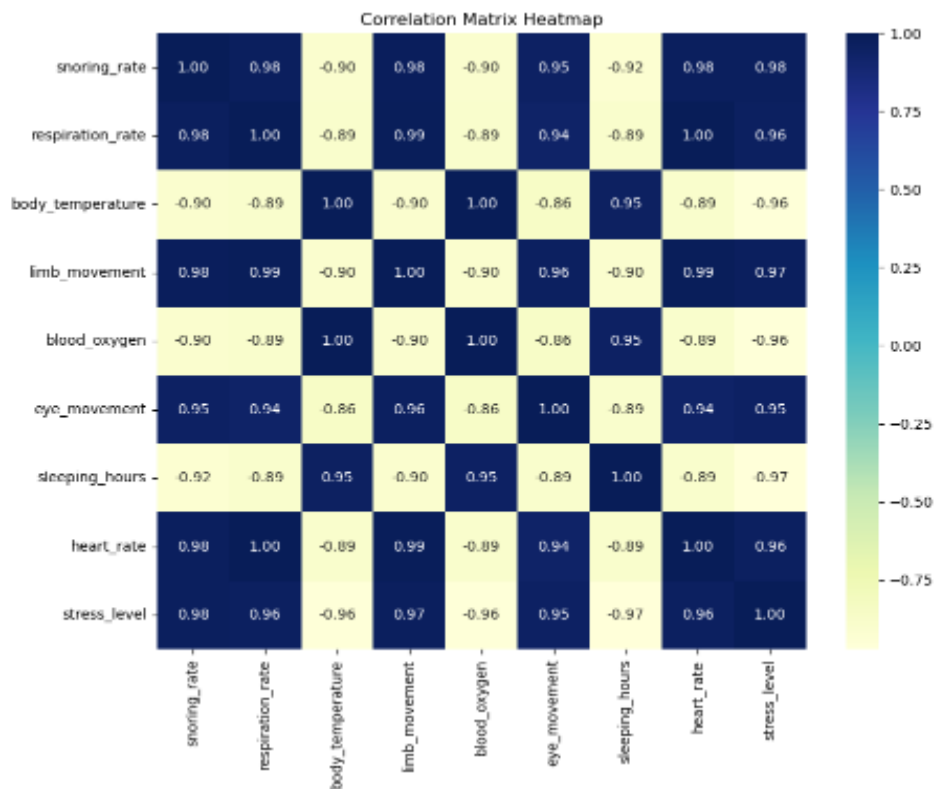
CHAPTER 4

EXPERIMENTAL RESULTS

4.1: FEATURE IMPORTANCE:



4.2: CORRELATION MATRIX:



- In this project, we apply three different machine learning models **Naive Bayes**, **Decision Tree**, and **Random Forest** to predict the stress level of individuals

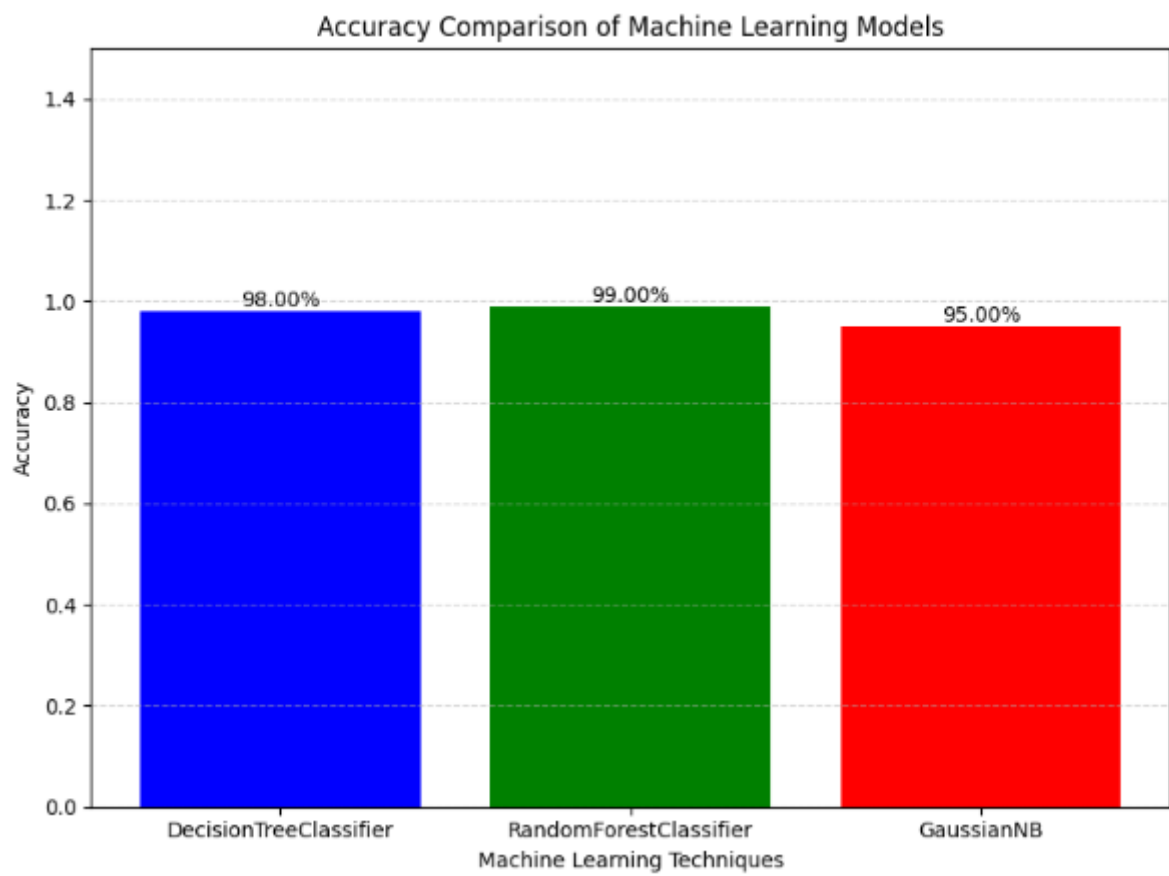
while they are sleeping. To evaluate the performance of these models, we use various quantitative evaluation metrics such as **accuracy, error rate, precision, recall, and F1-score**. These metrics allow us to assess the quality of the predictions made by each model.

- The results are presented in a tabular format, where each row corresponds to a different evaluation metric, and each column represents the performance of a specific model. This comparison enables us to determine which model performs best in terms of predicting stress levels accurately.

ALGORITHMS	ACCURACY	ERROR RATE
DECISION TREE	0.98	0.02
RANDOM FOREST	0.99	0.01
GAUSSIAN NAIVE BAYES	0.95	0.05

- Compare the machine learning algorithms like **Decision Tree, Random Forest and Naive bayes** Machine and choose the best one based on the **accuracy and error rate**. Which algorithm error rate is low this algorithm is best one. The **DT is 98%** and **RF is 99% accuracy, NB is the 95% accuracy**. And error rate is 0.02 for DT, 0.01 for RF, 0.05 for NB.
- So concluded that **RANDOM FOREST** is best algorithm for stress level prediction.

OUTPUT:



CHAPTER 5

CONCLUSION

The overall objective of the work is to predict more exactly the occurrence of stress levels using machine learning techniques. In this project, the Kaggle is used for performing the comparative analysis of three algorithms such as Random Forest, Decision Tree and Naive Bayes. From this project, it has been experimentally proven that Random Forest provides perfect results as compare to Decision Tree and Naïve Bayes.

CHAPTER 6

FUTURE SCOPE

In the future, the Stress prediction system will help monitor the stress levels of the given individual and help the individual in managing stress levels by real-time updates through mails. This will spread awareness regarding mental health and to manage or reduce stress during longer working periods.

CHAPTER 7

REFERENCES

- [1] Sarah Fallmann and Liming Chen. Computational sleep behavior analysis: A survey. *IEEE Access*, 7:142421–142440, 2019.
- [2] Mingmin Zhao, Shichao Yue, Dina Katabi, Tommi S Jaakkola, and Matt T Bianchi. Learning sleep stages from radio signals: A conditional adversarial architecture. In *International Conference on Machine Learning*, pages 4100–4109. PMLR, 2017.
- [3] Vanessa Ibañez, Josep Silva, and Omar Cauli. A survey on sleep ~ assessment methods. *PeerJ*, 6:e4849, 2018.
- [4] Nicolas Schaltenbrand, Regis Lengelle, M Toussaint, R Luthringer, ´ G Carelli, A Jacqmin, E Lainey, Alain Muzet, and Jean-Paul Macher. Sleep stage scoring using the neural network model: comparison between visual and automatic analysis in normal subjects and patients. *Sleep*, 19(1):26–35, 1996.
- [5] Chih-En Kuo, Yi-Che Liu, Da-Wei Chang, Chung-Ping Young, Fu-Zen Shaw, and Sheng-Fu Liang. Development and evaluation of a wearable device for sleep quality assessment. *IEEE Transactions on Biomedical Engineering*, 64(7):1547–1557, 2017.
- [6] Oded Z Maimon and Lior Rokach. *Data mining with decision trees: theory and applications*, volume 81. World scientific, 2014.
- [7] Priyanka and Dharmender Kumar. Decision tree classifier: A detailed survey. *International Journal of Information and Decision Sciences*, 12(3):246–269, 2020.
- [8] Quinlan, jr: C4. 5 programs for machine learning, morgan kaufmann, san mateo, california (1992).

- [9] BM Gayathri and CP Sumathi. An automated technique using gaussian naive bayes classifier to classify breast cancer. *International Journal of Computer Applications*, 148(6):16–21, 2016.
- [10] Leo Breiman. Random forests–random features. technical report 567, statistics department, university of california, berkeley,. 1999.
- [11] Leo Breiman. Bagging predictors. *Machine learning*, 24(2):123–140, 1996.
- [12] J Ross Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014.
- [13] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The elements of statistical learning: data mining, inference, and prediction, volume 2*. Springer, 2009. 18
- [14] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.
- [15] Sudarshan Nandy, Mainak Adhikari, Mohammad Ayoub Khan, Varun G Menon, and Sandeep Verma. An intrusion detection mechanism for secured iomt framework based on swarm-neural network. *IEEE Journal of Biomedical and Health Informatics*, 26(5):1969–1976,

CHAPTER 8

APPENDIX

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import StandardScaler

# Step 1: Load the dataset
data = pd.read_csv("D:\Education Content-1\generated_stress_data.csv")

# Step 2: Rename columns for better readability
data.columns = ['snoring_rate', 'respiration_rate', 'body_temperature',
                'limb_movement',
                'blood_oxygen', 'eye_movement', 'sleeping_hours', 'heart_rate',
                'stress_level']

# Step 3: Data overview
print("Rows and Columns of the dataset:", data.shape)
print(data.info())
print(data.describe(include="all"))
print("Null values per column:\n", data.isnull().sum())

# Step 4: Handle missing values
data.fillna(data.mean(), inplace=True)

# Step 5: Feature scaling
scaler = StandardScaler()
scaled_features = scaler.fit_transform(data.drop('stress_level', axis=1))
data_scaled = pd.DataFrame(scaled_features, columns=data.columns[:-1])

# Step 6: Splitting the data
X = data_scaled
```



```

y = data['stress_level']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Step 7: Initialize and train Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
dt_classifier.fit(X_train, y_train)

# Step 8: Make predictions and evaluate
y_pred_dt = dt_classifier.predict(X_test)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
error_rate_dt = 1 - accuracy_dt
report_dt = classification_report(y_test, y_pred_dt, zero_division=1)

print(f"Decision Tree Accuracy: {accuracy_dt:.2f}")
print(f"Error rate: {error_rate_dt:.2f}")
print("Decision Tree Classification Report:\n", report_dt)

# Step 9: Initialize and train Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42)
rf_classifier.fit(X_train, y_train)

# Step 10: Make predictions and evaluate
y_pred_rf = rf_classifier.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
error_rate_rf = 1 - accuracy_rf
report_rf = classification_report(y_test, y_pred_rf, zero_division=1)

print(f"Random Forest Accuracy: {accuracy_rf:.2f}")
print(f"Error rate: {error_rate_rf:.2f}")
print("Random Forest Classification Report:\n", report_rf)

# Step 11: Initialize and train Naive Bayes Classifier
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)

# Step 12: Make predictions and evaluate
y_pred_nb = nb_classifier.predict(X_test)

```

```

accuracy_nb = accuracy_score(y_test, y_pred_nb)
error_rate_nb = 1 - accuracy_nb
report_nb = classification_report(y_test, y_pred_nb, zero_division=1)

print(f"Naive Bayes Accuracy: {accuracy_nb:.2f}")
print(f"Error rate: {error_rate_nb:.2f}")
print("Naive Bayes Classification Report:\n", report_nb)

# Step 13: Plotting accuracies
models = ['Decision Tree', 'Random Forest', 'Naive Bayes']
accuracies = [accuracy_dt, accuracy_rf, accuracy_nb]

plt.figure(figsize=(8, 6))
plt.bar(models, accuracies, color=['blue', 'green', 'red'])
plt.title('Accuracy Comparison of Machine Learning Models')
plt.xlabel('Models')
plt.ylabel('Accuracy')

# Display accuracy values on top of the bars
for i in range(len(models)):
    plt.text(i, accuracies[i], f'{accuracies[i]*100:.2f}%', ha='center', va='bottom')

plt.ylim(0, 1) # Set y-axis limits
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

# Step 14: Predicting new data
new_data = pd.DataFrame([[90.0, 23.0, 92.0, 15.0, 90.0, 95.0, 2.0, 70.0]],
columns=X.columns)
scaled_new_data = scaler.transform(new_data) # Scale the new data
predicted_stress_level = rf_classifier.predict(scaled_new_data)

# Step 15: Map predicted stress level to labels
stress_level_labels = {0: "Low/Normal", 1: "Medium low", 2: "Medium", 3:
"Medium High", 4: "High"}
predicted_stress_label = stress_level_labels[predicted_stress_level]

```

```
# Provide feedback based on the predicted stress level
if predicted_stress_level == 0:
    print("Low stress level. Keep up the good work!")
elif predicted_stress_level == 1:
    print("Medium low stress level. Try to incorporate some relaxation techniques.")
elif predicted_stress_level == 2:
    print("Medium. Consider taking a break and engaging in stress-reducing
activities.")
elif predicted_stress_level == 3:
    print("Medium high stress level. It's important to prioritize self-care and seek
support.")
else:
```